Student Name: Shiva Sai Krishna S **Register Number:** 1911260318

Course: CSA0421 - Operating System **GitHub Repo:** krishshiva721/CSA04--Assignment1

TASK 5: SYSTEM CALL MECHANISM

Trap Execution, Parameter Passing, and Strace Profiling

1. STEP-BY-STEP SYSTEM CALL EXECUTION FLOW (READ)

1. **Application Invitation:** User program invokes the C standard library wrapper `read(fd, buf, count)`.
2. **Register Setup & Parameter Passing:** The library populates CPU architecture registers: System Call Number for `read` (e.g., `rax = 0` on x86_64), `rdi = fd`, `rsi = buf`, and `rdx = count`.
3. **Software Interrupt / Trap Execution:** The CPU executes a special instruction (e.g., `syscall` or `int 0x80`), triggering a mode switch.
4. **Mode Switch & Table Lookup:** CPU hardware switches execution mode from User Mode (Ring 3) to Kernel Mode (Ring 0), jumps to the Interrupt Vector Table, and enters `system_call()` handler.
5. **Kernel Dispatch & Execution:** Kernel validates virtual addresses, locates the VFS read routine, retrieves physical blocks from disk/buffer cache into the kernel page cache, and copies bytes to user space `buf`.
6. **Return to User Space:** Register `rax` receives the byte count returned. The `sysret/iret` instruction restores Ring 3 mode and returns control to user execution flow.

2. EXECUTION EVIDENCE VIA STRACE EXECUTION

```
$ gcc -O2 task5_test.c -o task5_test $ strace -e trace=openat,read,write,close ./task5_test openat(AT_FDCWD, "records.dat", O_RDONLY) = 3 read(3, "Patient ID: 1911260318\nStatus: "..., 4096) = 42 write(1, "Patient ID: 1911260318\nStatus: "..., 42) = 42 close(3) = 0 +++ exited with 0 +++
```

3. ALTERNATIVE CONCEPT & COMPLEXITY ANALYSIS

Alternative Scenario: Direct System Calls (open/read) vs. C Standard Library Buffering (fopen/fread)

For a file size of N bytes processed using a target buffer size b:

- **Direct System Call Approach (read):** Performs $K = \lceil N / b \rceil$ context switches. If $b = 1$ byte, reading a 1 MB file requires 1,048,576 trap instructions, causing severe CPU performance degradation.
- **Library Buffering (fread):** Employs an internal user-space ring buffer (typically $B = 4096$ bytes). The library executes only $K_{lib} = \lceil N / 4096 \rceil$ true system traps regardless of small read call granularities.
- **Time/Space Complexity Trade-off:** Direct unbuffered read incurs total context switch overhead $Time = O(N / b)$. Standard `fread` reduces time complexity to $Time = O(N / B)$ at the cost of a static memory allocation $Space = O(B)$ per stream handle.